Course: AI Assisted Coding (23CS002PC304)

Name : P. Samanvith
Batch No : 40
HT No : 2303A52090

Task 1: Student Academic Data Cleaning

Task Description

Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

- Handle missing values in marks, attendance, and department.
- Remove duplicate student records based on student ID.
- Standardize text fields such as department names.
- Encode categorical variables like gender and department.

Prompt Used

> "Write a Python script using pandas to clean a student academic dataset ('students.csv'). Handle missing values in 'marks', 'attendance' (using median/mean), and 'department' (using mode). Remove duplicate rows based on 'student_id'. Standardize the 'department' column by stripping whitespaces and converting to uppercase. Finally, use Label Encoding to encode the 'gender' and 'department' categorical variables."

Python Code

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# 1. Load the dataset
# data = pd.read_csv("students.csv")
# Using simulated data for demonstration purposes
data = pd.DataFrame({
   'student_id': [101, 102, 103, 101, 104, 105],
   'gender': ['Male', 'Female', 'Female', 'Male', 'Male', 'Female'],
   'department': [' cse ', 'ECE', None, ' cse ', 'MECH', 'ece'],
   'marks': [85, 90, None, 85, 75, 88],
   'attendance': [90, None, 85, 90, 80, 95]
})
```

```
print("--- Original Data ---")
print(data.head())

# 2. Remove duplicate records based on student ID
data = data.drop_duplicates(subset=['student_id'])

# 3. Handle missing values
# Fill numerical missing values with median
data['marks'] = data['marks'].fillna(data['marks'].median())
data['attendance'] = data['attendance'].fillna(data['attendance'].median())
# Fill categorical missing values with mode
data['department'] = data['department'].fillna(data['department'].mode()[0])

# 4. Standardize text fields (department names)
# Remove leading/trailing spaces and convert to uppercase
data['department'] = data['department'].str.strip().str.upper()

# 5. Encode categorical variables
le_gender = LabelEncoder()
le_dept = LabelEncoder()

data['gender_encoded'] = le_gender.fit_transform(data['gender'])
data['department_encoded'] = le_dept.fit_transform(data['department'])

# Drop original categorical columns if desired (optional)
# data = data.drop(['gender', 'department'], axis=1)

print("\n--- Cleaned and Encoded Data ---")
print(data.head())
```

Output

```
--- Original Data ---
  student_id  gender department  marks  attendance
0        101    Male     cse     85.0       90.0
1        102  Female       ECE   90.0        NaN
2        103  Female      None    NaN       85.0
3        101    Male     cse     85.0       90.0
4        104    Male      MECH   75.0       80.0
```

--- Cleaned and Encoded Data ---

	student_id	gender	department	marks	attendance	gender_encoded	department_encoded
0	101	Male	CSE	85.0	90.0	1	0
1	102	Female	ECE	90.0	87.5	0	1
2	103	Female	CSE	86.5	85.0	0	0
4	104	Male	MECH	75.0	80.0	1	2
5	105	Female	ECE	88.0	95.0	0	1

Task 2: Financial Transaction Data Preprocessing

Task Description

Preprocess financial transaction data using AI-generated Python code.

Instructions:

- Convert transaction date columns to datetime format.
- Create derived features such as transaction month and year.
- Normalize the transaction amount column.
- Identify and handle extreme transaction values.

Prompt Used

> "Write a Python script using pandas and sklearn to preprocess a financial transactions dataset. Convert 'transaction_date' to datetime format, and extract 'month' and 'year' into separate columns. Handle outliers in 'transaction_amount' using the Interquartile Range (IQR) method. Finally, normalize the 'transaction_amount' column using MinMaxScaler."

Python Code

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 1. Load the dataset
# transactions = pd.read_csv("transactions.csv")
# Simulated data
transactions = pd.DataFrame({
   'transaction_id': [1, 2, 3, 4, 5],
   'transaction_date': ['2023-01-15', '2023-02-20', '2023-03-05', '2023-04-12', '2023-05-25'],
   'transaction_amount': [150.0, 200.5, 9999.0, 50.0, 175.2] # 9999 is an outlier
```

```
})

print("--- Original Data Info ---")
transactions.info()

# 2. Convert transaction date to datetime format
transactions['transaction_date'] = pd.to_datetime(transactions['transaction_date'])

# 3. Create derived features (month and year)
transactions['transaction_month'] = transactions['transaction_date'].dt.month
transactions['transaction_year'] = transactions['transaction_date'].dt.year

# 4. Identify and handle extreme transaction values (Outliers using IQR)
Q1 = transactions['transaction_amount'].quantile(0.25)
Q3 = transactions['transaction_amount'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Capping the outliers to the upper and lower bounds
transactions['transaction_amount'] = np.where(
   transactions['transaction_amount'] > upper_bound, upper_bound,
   np.where(transactions['transaction_amount'] < lower_bound, lower_bound,
transactions['transaction_amount'])
)

# 5. Normalize the transaction amount column
scaler = MinMaxScaler()
transactions['transaction_amount_normalized'] =
scaler.fit_transform(transactions[['transaction_amount']])

print("\n--- Processed Dataset ---")
print(transactions)
```

Output

```
--- Original Data Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
```

```
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   transaction_id     5 non-null     int64
 1   transaction_date   5 non-null     object
 2   transaction_amount  5 non-null     float64
dtypes: float64(1), int64(1), object(1)
memory usage: 248.0+ bytes

--- Processed Dataset ---
  transaction_id transaction_date  transaction_amount  transaction_month  transaction_year
transaction_amount_normalized
0              1      2023-01-15             150.000                  1              2023                      0.442229
1              2      2023-02-20             200.500                  2              2023                      0.665561
2              3      2023-03-05             276.125                  3              2023                      1.000000
3              4      2023-04-12              50.000                  4              2023                      0.000000
4              5      2023-05-25             175.200                  5              2023                      0.553697
```

Task 3: Environmental Sensor Data Preparation

Task Description

Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- Handle missing values in temperature, humidity, and air_quality_index.
- Scale numerical features using standard scaling.
- Encode categorical fields such as sensor_status.
- Split the dataset into training and testing subsets.

Prompt Used

> "Write a Python script using pandas and sklearn for environmental sensor data preparation. Fill missing values in 'temperature', 'humidity', and 'air_quality_index' using the mean. Scale these numerical features using StandardScaler. Encode the 'sensor_status' column using LabelEncoder. Finally, split the dataset into training and testing sets (80/20 split)."

Python Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
# 1. Load the dataset
# data = pd.read_csv("sensor_data.csv")
# Simulated data
data = pd.DataFrame({
   'temperature': [22.5, np.nan, 25.1, 19.8, 24.0, 23.5],
   'humidity': [45, 50, np.nan, 55, 48, 42],
   'air_quality_index': [30, 45, 50, np.nan, 35, 40],
   'sensor_status': ['Active', 'Inactive', 'Active', 'Active', 'Maintenance', 'Active']
})

# 2. Handle missing values
num_cols = ['temperature', 'humidity', 'air_quality_index']
for col in num_cols:
   data[col] = data[col].fillna(data[col].mean())

# 3. Encode categorical fields
le = LabelEncoder()
data['sensor_status_encoded'] = le.fit_transform(data['sensor_status'])

# 4. Scale numerical features using standard scaling
scaler = StandardScaler()
data[num_cols] = scaler.fit_transform(data[num_cols])

# 5. Split the dataset into training and testing subsets
X = data[['temperature', 'humidity', 'air_quality_index', 'sensor_status_encoded']]
y = data['sensor_status_encoded'] # Assuming status is the target for demonstration

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("--- Preprocessed Training Features (X_train) ---")
print(X_train)
print(f"\nTraining set shape: {X_train.shape}, Testing set shape: {X_test.shape}")
```

Output

```
--- Preprocessed Training Features (X_train) ---
  temperature  humidity  air_quality_index  sensor_status_encoded
5    0.301385 -1.503437         0.000000                      0
2    1.192534  0.000000         1.511858                      0
```

```
4    0.579868 -0.000000       -0.755929                 2
3   -1.787948  1.754010        0.000000                 0
```

Training set shape: (4, 4), Testing set shape: (2, 4)

Task 4: Real-Time Application: Smart City Traffic Data Cleaning

Task Description

A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- Standardize road and location names.
- Fill missing traffic density values using appropriate statistical methods.
- Remove duplicate sensor readings.
- Normalize speed and vehicle count metrics.
- Generate a brief summary comparing dataset quality before and after cleaning.

Prompt Used

> "Write a Python script to clean smart city traffic data. Standardize 'road_name' and 'location' by making them lowercase and stripping whitespace. Fill missing 'traffic_density' values with the median. Remove duplicate rows. Normalize 'speed' and 'vehicle_count' using MinMaxScaler. Print a data quality summary showing row counts and missing values before and after cleaning."

Python Code

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 1. Load data
# traffic = pd.read_csv("traffic_data.csv")
traffic = pd.DataFrame({
   'sensor_id': [1, 2, 1, 3, 4],
   'road_name': [' Main St ', 'broadway', ' Main St ', 'ELM ST', 'broadway'],
   'location': ['North', ' South ', 'North', 'East', 'South'],
   'traffic_density': [75.5, np.nan, 75.5, 40.2, 88.0],
   'speed': [45, 30, 45, 55, np.nan],
```

```
    'vehicle_count': [120, 85, 120, 60, 200]
})

# --- Before Cleaning Summary ---
initial_rows = len(traffic)
initial_missing = traffic.isnull().sum().sum()

# 2. Standardize road and location names
traffic['road_name'] = traffic['road_name'].str.lower().str.strip()
traffic['location'] = traffic['location'].str.lower().str.strip()

# 3. Remove duplicate sensor readings
traffic = traffic.drop_duplicates()

# 4. Fill missing traffic density and speed values
traffic['traffic_density'] = traffic['traffic_density'].fillna(traffic['traffic_density'].median())
traffic['speed'] = traffic['speed'].fillna(traffic['speed'].median())

# 5. Normalize speed and vehicle count metrics
scaler = MinMaxScaler()
traffic[['speed_norm', 'vehicle_count_norm']] = scaler.fit_transform(traffic[['speed',
'vehicle_count']])
traffic = traffic.drop(['speed', 'vehicle_count'], axis=1)

# --- After Cleaning Summary ---
final_rows = len(traffic)
final_missing = traffic.isnull().sum().sum()

print("=== Data Quality Summary ===")
print(f"Before Cleaning: {initial_rows} rows, {initial_missing} missing values.")
print(f"After Cleaning : {final_rows} rows, {final_missing} missing values.")
print("\n--- Cleaned Traffic Data ---")
print(traffic)
```

Output

```
=== Data Quality Summary ===
Before Cleaning: 5 rows, 2 missing values.
After Cleaning : 4 rows, 0 missing values.
```

--- Cleaned Traffic Data ---

	sensor_id	road_name	location	traffic_density	speed_norm	vehicle_count_norm
0	1	main st	north	75.50	0.6	0.428571
1	2	broadway	south	75.50	0.0	0.178571
3	3	elm st	east	40.20	1.0	0.000000
4	4	broadway	south	88.00	0.6	1.000000

Task 5: Social Media Text Data Cleaning and Feature Extraction

Task Description

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- Remove duplicate posts and null text entries.
- Clean text by removing URLs, special characters, and extra spaces.
- Convert text to lowercase and perform basic tokenization.
- Remove stopwords and perform stemming or lemmatization.
- Generate basic text features such as word count and sentiment score.

Prompt Used

> "Write a Python script using pandas, re, nltk, and textblob to clean social media posts. Drop duplicates and null values. Clean the 'text' column by removing URLs, special characters, and extra spaces. Convert the text to lowercase, tokenize it, remove stopwords, and apply WordNetLemmatizer. Finally, extract features like 'word_count' and a 'sentiment_score' using TextBlob."

Python Code

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from textblob import TextBlob

# Download required NLTK data (uncomment if running for the first time)
```

```
# nltk.download('punkt')
# nltk.download('stopwords')
# nltk.download('wordnet')

# 1. Load data
# posts = pd.read_csv("social_media_posts.csv")
posts = pd.DataFrame({
    'post_id': [1, 2, 3, 1, 4],
    'text': [
        "I love the new update! 😊 Check it out at [http://link.com](http://link.com)",
        "Terrible experience... my app keeps crashing. #angry",
        None,
        "I love the new update! 😊 Check it out at [http://link.com](http://link.com)", # Duplicate
        "   Data cleaning is SO much FUN!!!   "
    ]
})

# 2. Remove duplicate posts and null text entries
posts = posts.dropna(subset=['text'])
posts = posts.drop_duplicates(subset=['text'])

# Initialize Lemmatizer and Stopwords
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def clean_text(text):
    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    # Remove special characters and numbers (keep only letters)
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    # Convert to lowercase and remove extra spaces
    text = text.lower().strip()
    text = re.sub(r'\s+', ' ', text)

    # Tokenization
    tokens = word_tokenize(text)

    # Remove stopwords and Lemmatize
    cleaned_tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in
stop_words]
```

```
    return " ".join(cleaned_tokens)

# Apply text cleaning
posts['cleaned_text'] = posts['text'].apply(clean_text)

# 3. Generate basic text features
# Word Count
posts['word_count'] = posts['cleaned_text'].apply(lambda x: len(x.split()))

# Sentiment Score (-1.0 to 1.0)
posts['sentiment_score'] = posts['cleaned_text'].apply(lambda x:
TextBlob(x).sentiment.polarity)

print("--- Final Cleaned Data & Features ---")
print(posts[['cleaned_text', 'word_count', 'sentiment_score']])
```

Output

```
--- Final Cleaned Data & Features ---
            cleaned_text  word_count  sentiment_score
0         love new update           3         0.318182
1  terrible experience app keeps crashing  5        -1.000000
4        data cleaning much fun     4         0.300000
```